

Obstacles for a Llama

Լաման ուզում է ճանապարհորդել Անդյան սարահարթով: Այն ունի սարահարթի քարտեզ $N \times M$ քառակուսի վանդակավոր ցանցի տեսքով: Քարտեզի տողերը համարակալված են 0 -ից մինչև $N - 1$ ՝ վերևից ներքև, իսկ սյուները համարակալված են 0 -ից մինչև $M - 1$ ՝ ձախից աջ: Քարտեզի i տողում և j սյունակում գտնվող վանդակը ($0 \leq i < N, 0 \leq j < M$) նշանակենք (i, j) -ով:

Լաման ուսումնասիրել է սարահարթի կլիման և հայտնաբերել, որ քարտեզի յուրաքանչյուր տողի բոլոր վանդակներն ունեն նույն **ջերմաստիճանը**, և քարտեզի յուրաքանչյուր սյունակի բոլոր վանդակներն ունեն նույն **խոնավությունը**: Լաման ձեռք տվել է երկու ամբողջ թվերի զանգված՝ T և H համապատասխանաբար N և M երկարություններով: Այստեղ $T[i]$ ($0 \leq i < N$)-ը ցույց է տալիս i տողի վանդակների ջերմաստիճանը, իսկ $H[j]$ ($0 \leq j < M$)-ը՝ j սյունակի վանդակների խոնավությունը:

Լաման նաև ուսումնասիրել է սարահարթի բուսական աշխարհը և նկատել, որ (i, j) վանդակը **զերծ է բուսականությունից** այն և միայն այն դեպքում, եթե նրա ջերմաստիճանը մեծ է խոնավությունից, այսինքն $T[i] > H[j]$:

Լաման կարող է սարահարթով անցնել միայն **վազեր ուղիներով**: Վազեր ուղին տարբեր վանդակների հաջորդականություն է, որը բավարարում է հետևյալ պայմաններին.

- Ճանապարհի հաջորդական վանդակների յուրաքանչյուր զույգ ունի ընդհանուր կողմ:
- Ճանապարհի բոլոր վանդակները զերծ են բուսականությունից:

Ձեր խնդիրն է պատասխանել Q հարցումների: Յուրաքանչյուր հարցման համար ձեռք տրվում է չորս ամբողջ թիվ՝ L, R, S և D : Դուք պետք է որոշեք, թե արդյոք գոյություն ունի վազեր ուղի, որը

- սկսվում է $(0, S)$ վանդակից և ավարտվում $(0, D)$ վանդակում:
- Ճանապարհի բոլոր վանդակները գտնվում են L ից մինչև R սյուների սահմաններում՝ ներառյալ:

Երաշխավորված է, որ $(0, S)$, և $(0, D)$ վանդակները զերծ են բուսականությունից:

Իրականացման մանրամասներ

Առաջին ֆունկցիան, որը դուք պետք է իրականացնեք, հետևյալն է.

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- $T : N$ երկարությամբ զանգված, որը ցույց է տալիս յուրաքանչյուր տողում ջերմաստիճանը:
- $H : M$ երկարությամբ զանգված, որը ցույց է տալիս յուրաքանչյուր սյունակում խոնավությունը:
- Այս ֆունկցիան կանչվում է յուրաքանչյուր թեստի համար ճիշտ մեկ անգամ `can_reach`-ի բոլոր կանչերից առաջ:

Երկրորդ ֆունկցիան, որը դուք պետք է իրականացնեք, հետևյալն է.

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : հարցումը նկարագրող ամբողջ թվեր:
- Այս ֆունկցիան կանչվում է Q անգամ յուրաքանչյուր թեստի համար:

Այս ֆունկցիան պետք է վերադարձնի `true` այն և միայն այն դեպքում, եթե գոյություն ունի վազեր ուղի $(0, S)$ վանդակից դեպի $(0, D)$ վանդակն այնպես, որ ուղու բոլոր վանդակները գտնվեն L մինչև R սյուների սահմաններում ներառյալ:

Սահմանափակումներ

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq T[i] \leq 10^9$ յուրաքանչյուր i համար, այնպես որ $0 \leq i < N$:
- $0 \leq H[j] \leq 10^9$ յուրաքանչյուր j համար, այնպես որ $0 \leq j < M$:
- $0 \leq L \leq R < M$
- $L \leq S \leq R$
- $L \leq D \leq R$
- Երկու $(0, S)$ և $(0, D)$ վանդակներն էլ զերծ են բուսականությունից:

Ենթախնդիրներ

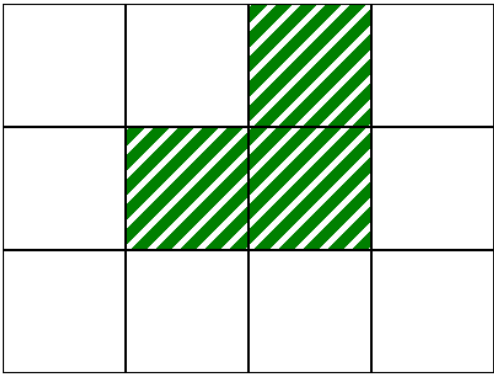
Ենթախնդիր	Միավոր	Լրացուցիչ սահմանափակումներ
1	10	$L = 0, R = M - 1$ յուրաքանչյուր հարցման համար: $N = 1$.
2	14	$L = 0, R = M - 1$ յուրաքանչյուր հարցման համար: $T[i - 1] \leq T[i]$ յուրաքանչյուր i համար այնպես, որ $1 \leq i < N$.
3	13	$L = 0, R = M - 1$ յուրաքանչյուր հարցման համար: $N = 3$ և $T = [2, 1, 3]$:
4	21	$L = 0, R = M - 1$ յուրաքանչյուր հարցման համար: $Q \leq 10$.
5	25	$L = 0, R = M - 1$ յուրաքանչյուր հարցման համար:
6	17	Լրացուցիչ սահմանափակումներ չկան:

Օրինակ

Դիտարկենք հետևյալ կանչը.

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

Սա համապատասխանում է հետևյալ պատկերում տրված քարտեզին, որտեղ սպիտակ վանդակները զերծ են բուսականությունից:

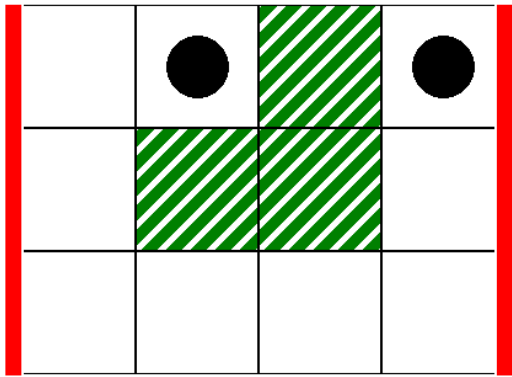


Որպես առաջին հարցում, դիտարկենք հետևյալ կանչը.

```
can_reach(0, 3, 1, 3)
```

Սա համապատասխանում է հետևյալ պատկերում պատկերված սցենարին, որտեղ հաստ ուղղահայաց գծերը ցույց են տալիս սյուների միջակայքը $L = 0$ -ից մինչև $R = 3$,

իսկ սև շրջանները՝ սկզբնական և վերջնական վանդակները:



Այս դեպքում լաման կարող է հասնել (0,1) վանդակից (0,3) վանդակ հետևյալ վավեր ուղով՝

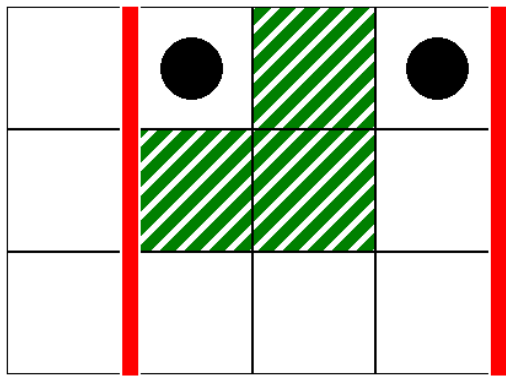
(0,1), (0,0), (1,0), (2,0), (2,1), (2,2), (2,3), (1,3), (0,3)

Հետևաբար, այս կանչը պետք է վերադարձնի `true` :

Որպես երկրորդ հարցում, դիտարկենք հետևյալ կանչը.

```
can_reach(1, 3, 1, 3)
```

Սա համապատասխանում է հետևյալ նկարում պատկերված սցենարին.



Այս դեպքում, (0,1) վանդակից դեպի (0,3) վանդակ վավեր ուղի չկա, այնպես որ ուղղու բոլոր վանդակները գտնվում են 1 մինչև 3 սյուների սահմաններում, ներառյալ: Հետևաբար, այս կանչը պետք է վերադարձնի `false` :

Գրեյդերի նմուշ

Մուտքաային տվյալներ

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

Այստեղ, $L[k], R[k], S[k]$ և $D[k]$ ($0 \leq k < Q$)-ը սահմանում են `can_reach` ֆունկցիայի յուրաքանչյուր կանչի պարամետրերը:

Ելքային տվյալներ

```
A[0]
A[1]
...
A[Q-1]
```

Այստեղ, $A[k]$ ($0 \leq k < Q$)-ը 1 է, եթե `can_reach(L[k], R[k], S[k], D[k])` վերադարձրեց `true`, իսկ հակառակ դեպքում 0 է: